

```
* checking dependencies in R code ... OK
* checking compilation flags in Makevars ... OK
* checking for GNU extensions in Makefiles ... OK
* checking for portable use of $(BLAS_LIBS) and $(LAPACK_
LIBS) ... OK
* checking use of PKG_*FLAGS in Makefiles ... OK
* checking include directives in Makefiles ... OK
* checking compiled code ... OK
* checking examples ... OK
* checking for unstated dependencies in 'tests' ... OK
* checking tests ...
  Running 'doRUnit.R'
OK
* DONE
```

Status: OK

testthat

testthat is another unit testing framework similar to the xUnit family of test packages. You can install the same in an R session using the following command:

```
> install.packages("testthat")
```

The library needs to be loaded into the R session, as shown below:

```
> library(testthat)
```

Consider the same square function example:

```
> square <- function(x) return (x * x)
```

You can use the testthat library's *expect_equal* assertion to validate the function execution:

```
> test_that("square of two", {
  expect_equal(square(2), 4)
})
Test passed
```

An error is reported if there is a mismatch in the *expect_equal* check, as shown below:

```
> expect_equal(square(2), 3)
Error: square(2) not equal to 3.
1/1 mismatches
[1] 4 - 3 == 1
```

The *expect_equal()* function is more suitable for numeric values only. The *expect_identical* function is useful to compare objects, and checks if they have the same values as

well. A couple of examples are shown below:

```
> u <- c('one', 'two', 'three')
> v <- c('one', 'two', 'three')
> a <- list(1, 2, 3, 4)

> test_that("expect identical", {
  expect_identical(u, a)
})
— Failure (Line 2): expect identical —
`u` not identical to `a`.
Types not compatible: character is not list
Error: Test failed

> test_that("expect identical", {
  expect_identical(u, v)
})
Test passed
```

The *expect_is* function checks if the argument belongs to an object or inherits it from a specific class. For example:

```
> test_that("expect is", {
  expect_is(u, "character")
})
Test passed

> test_that("expect is", {
  expect_is(a, "character")
})
— Failure (Line 2): expect is —
`a` inherits from `list` not `character`.
Error: Test failed
```

A character vector can be matched to a regular expression using the *expect_match* function. A few examples are given below:

```
> quote <- "Science is curiosity, testing and experimenting"
> test_that("expect match", {
  expect_match(quote, "Science")
})
Test passed

> test_that("expect match", {
  expect_match(quote, "science")
})
— Failure (Line 2): expect match —
`quote` does not match "science".
Actual value: "Science is curiosity, testing and
```